Design Document

Accelerated Mobile Video Processing System

Real-Time Camera Pipeline using SIMD and Mobile GPU

Team: Abhinav Gupta (2025MCS2089)
Pratik Chaudhari (2025MCS2096)

Course: COP 7001 — Cornerstone Project

Date: March 20, 2026

Repository: github.com/rgabhi/accelerated_video_processing_system

Timeline: 80 Days (4 Phases)

COP 7001

Phase 1 (Day 1–20): Completed ✓ **Phase 2 (Day 21–40): Completed ✓**

Phase 3 (Day 41–70): In Progress **Phase 4 (Day 71–80): Upcoming**

Contents

1 Executive Summary

AccelerateApp is a real-time Android camera pipeline that applies compute-intensive pixel-level transformations to live video frames. The project serves as a rigorous performance benchmarking platform for heterogeneous mobile compute, comparing four execution tiers:

1. **Baseline** — Scalar Kotlin `for` loops (performance floor).
2. **SIMD** — Vectorized C++ using ARM NEON intrinsics (4–16 pixels per instruction).
3. **GPU** — OpenGL ES 3.1 Compute Shaders (massively parallel).
4. **Hybrid** — Concurrent SIMD + GPU on split frame regions.

Current Status

Phases 1 and 2 are **complete**. The SIMD pipeline delivers a **2–2.4× speedup** over baseline across five production filters (Sepia, Gaussian Blur, Sobel Edge, Emboss, Vignette). Sobel edge detection runs at the **camera sensor's hardware maximum of 30 FPS**, confirming the SIMD code now outpaces the input data rate.

2 Problem Analysis

Modern mobile SoCs pack an extraordinary variety of compute units — ARM big.LITTLE CPU cores, a 128-bit NEON SIMD engine, a multi-core GPU, and dedicated DSPs. Yet most application code remains single-threaded and scalar, leaving vast silicon resources idle.

We chose real-time image filtering because it is **embarrassingly parallel** at the pixel level, making it an ideal workload to stress-test data-level parallelism (SIMD) against thread-level parallelism (GPU).

2.1 Core Sub-Problems

#	Sub-Problem	Our Approach
P1	Camera Pipeline	CameraX ImageAnalysis with `STRATEGY_KEEP_ONLY_LATEST` backpressure.
P2	Baseline Processing	Pure Kotlin scalar filters — correctness oracle for numerical verification.
P3	SIMD Acceleration	C++ via JNI with ARM NEON: structural loads, fixed-point math, bitwise kernels.
P4	GPU Acceleration	OpenGL ES 3.1 Compute Shaders with SSBO-based pixel I/O.
P5	Hybrid Coordination	Horizontal frame split — top half to NEON thread, bottom half to GPU.

Table 1: The five core sub-problems guiding the implementation.

3 System Architecture

3.1 Three-Stage Pipeline

The processing pipeline is modeled as three interchangeable stages, mapped to physical hardware based on the user-selected execution mode.

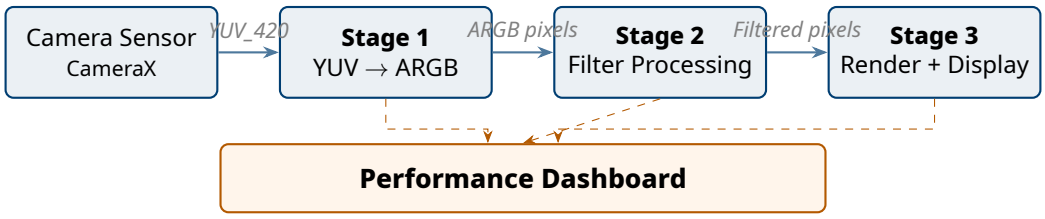

Figure 1: Three-stage pipeline with timing instrumentation feeding the live dashboard.

3.2 NEON Register Architecture

A key insight during Phase 2 was understanding how pixel data maps to ARM NEON hardware registers. NEON provides 128-bit vector registers that can be partitioned into different lane widths:

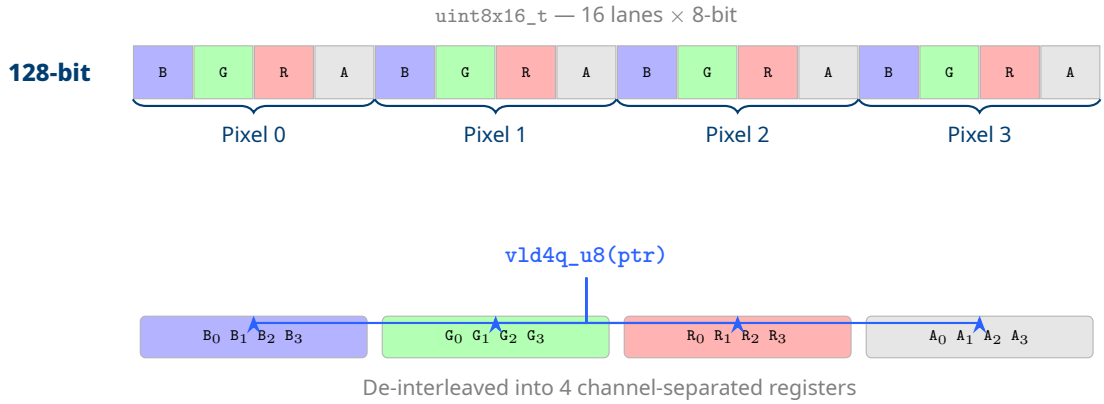

Figure 2: ARM NEON 128-bit register layout. `vld4q_u8` loads 4 pixels and de-interleaves into separate B, G, R, A vectors in hardware.

3.3 Data Flow Format

The camera outputs `YUV_420_888`. We normalize this immediately in Stage 1 into packed `ARGB_8888` (`IntArray`, 4 bytes/pixel). All filters — whether Kotlin, C++, or GLSL — operate on this unified format.

4 Technology Stack

Component	Decision & Rationale
Application	**Kotlin + Jetpack Compose**. Declarative, reactive UI with automatic state-driven re-renders.
Native Bridge	**C++17 via JNI**. Required for ARM NEON intrinsic access; `GetIntArrayElements` for safe memory pinning.
SIMD Engine	**ARM NEON** (`arm_neon.h`). 128-bit registers processing 4–16 pixels per instruction.
GPU API	**OpenGL ES 3.1 Compute Shaders**. Preferred over Vulkan — less boilerplate, sufficient shared memory and barrier support.
Camera	**CameraX**. Handles OEM sensor quirks; `STRATEGY_KEEP_ONLY_LATEST` provides natural backpressure.
Build	**CMake + Gradle**. NDK's official native build system.

Table 2: Core technology decisions.

5 Filter Workloads

The five filters span a spectrum from simple per-pixel arithmetic to heavy spatial convolutions, allowing us to observe how each execution tier responds to increasing computational density.

Filter	Kernel	Workload	SIMD Strategy
Sepia	1×1	BT.601 grayscale + channel bias. Light arithmetic.	`vmulq_f32` + `vmlaq_f32` on 4 pixels.
Gaussian Blur	5×5	Heavy spatial convolution. Memory-bandwidth bound.	Separable 1D passes + bitwise kernel math (`vshlq`).
Sobel Edge	3×3	Two-pass: grayscale then gradient magnitude.	`vld4q_u8` structural loads + `vabdq_u16` absolute diff.
Emboss	3×3	Directional convolution + bias of 128.	`vmlaq_s32` multiply-accumulate on 4 pixels.
Vignette	1×1	Per-pixel distance from center. Math-heavy.	`vrsqrteq_f32` hardware reciprocal square root.

Table 3: Filter specifications and corresponding SIMD optimization strategies.

6 Phase 1: Pipeline & Baseline (Day 1–20) ✓

Phase 1 established the foundational pipeline and baseline performance floor.

Key Achievements:

- **CameraX Integration:** Stable image analysis pipeline with proper device rotation handling.

- **Backpressure:** `STRATEGY_KEEP_ONLY_LATEST` drops stale frames to prevent latency spiraling.
- **Buffer Pooling:** Pre-allocated `IntArray` and mutable `Bitmap` to eliminate GC churn (sawtooth allocation pattern resolved).
- **Baseline Metrics:** Gaussian blur in pure Kotlin: ~10 FPS. Confirmed benchmarking instrumentation accuracy.
- **Dashboard:** Real-time overlay displaying FPS, latency, mode indicator, and sparkline graph.

7 Phase 2: ARM NEON Acceleration (Day 21–40) ✓

Phase 2 was the most technically intensive phase. What began as a straightforward port of Kotlin filters to C++ NEON intrinsics evolved into a multi-round performance engineering exercise.

7.1 The Optimization Journey

The SIMD implementation went through **four distinct optimization rounds**, each driven by profiling and diagnosis of the previous round's bottlenecks.

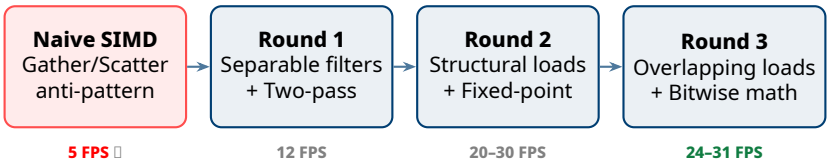

Figure 3: Four optimization rounds for Gaussian blur FPS progression. Initial SIMD was *slower* than baseline due to the gather/scatter anti-pattern.

7.2 The Gather/Scatter Anti-Pattern

The initial SIMD implementation was **slower than the Kotlin baseline**. The root cause: manually packing pixel data byte-by-byte into arrays before loading them into NEON registers.

The Fundamental Rule of SIMD

Data must be **contiguous in memory**. If the CPU must search around memory to collect individual bytes before vector math, the gathering overhead *destroys* any parallelism benefit.

The fix required *re-architecting the algorithms* to match the hardware's data access patterns, not forcing the hardware to run scalar-shaped code.

7.3 Key Optimization Strategies

7.3.1 Separable Gaussian Blur

The 2D 5×5 Gaussian kernel is **linearly separable**, decomposable into two 1D passes:

$$\underbrace{\begin{bmatrix}1 & 4 & 6 & 4 & 1\end{bmatrix}}_{\text{Horizontal}} \times \underbrace{\begin{bmatrix}1\\4\\6\\4\\1\end{bmatrix}}_{\text{Vertical}}$$

This reduces operations from 25 multiply-adds to 10 per pixel per channel — a **60% reduction**.

The kernel weights {1, 4, 6, 4, 1} sum to 16 (a power of 2), enabling **pure bitwise arithmetic**:

```
// x * 1: identity
uint16x8_t sum = vmovl_u8(p_m2.val[c]);
// x * 4: shift left 2
sum = vaddq_u16(sum, vshlq_n_u16(vmovl_u8(p_m1.val[c]), 2));
// x * 6: (shift left 2) + (shift left 1)
uint16x8_t center = vmovl_u8(p_0.val[c]);
sum = vaddq_u16(sum, vaddq_u16(
    vshlq_n_u16(center, 2), vshlq_n_u16(center, 1)));
// / 16: shift right 4, narrow u16 -> u8
out.val[c] = vshrn_n_u16(sum, 4);
```

Listing 1: Gaussian kernel using only bit-shifts (zero multiplications).

7.3.2 Fixed-Point Grayscale (Sobel Pass 1)

Floating-point conversions are expensive on mobile. **Fixed-point arithmetic** eliminates them entirely:

$$\text{Gray}_{\text{float}} = 0.299R + 0.587G + 0.114B \quad \Longrightarrow \quad \text{Gray}_{\text{int}} = \frac{77R + 150G + 29B}{256}$$

Where $\div 256 = >> \ 8$ (single-cycle bit-shift). This processes **16 pixels per iteration** using `vld4q_u8` structural loads.

7.3.3 Vectorized Sobel (Pass 2)

The Sobel convolution on the 1-byte-per-pixel grayscale buffer uses **overlapping loads** to exploit L1 cache:

```
// vabdq_u16: |a - b| in a single hardware instruction
uint16x8_t abs_gx = vabdq_u16(col_right, col_left);
uint16x8_t abs_gy = vabdq_u16(row_bottom, row_top);
// Saturating narrow: clamps to 255 automatically
uint8x8_t mag = vqmovn_u16(vaddq_u16(abs_gx, abs_gy));
```

Listing 2: Sobel magnitude via absolute difference — one instruction.

7.4 Performance Results

Round	Gaussian (FPS)	Sobel (FPS)	Key Technique
Baseline (Kotlin)	~10	~15	Scalar `for` loops
Naive SIMD	~5 ↓	~10 ↓	Gather/scatter anti-pattern
Round 1	~12	~20	Separable + two-pass
Round 2	—	~30 ✓	Structural loads + fixed-point
Round 3 (Final)	**~24**	**~31** ✓	Overlapping loads + bitwise

Table 4: FPS progression across four optimization rounds. Sobel is hardware-capped by the camera sensor at 30 FPS.

7.5 Understanding the Performance Ceilings

Sobel at 30 FPS: Camera Sensor Limit

The Sobel filter processes each frame in ~10 ms, but CameraX delivers frames at 30 FPS (33.3 ms intervals). The CPU **outpaces the camera sensor**. Breaking past 30 FPS requires reconfiguring to 60 FPS mode via Camera2 Interop.

Gaussian at 24 FPS: The Memory Wall

The separable Gaussian requires an 8 MB temporary ARGB buffer for 1080p frames. This **exceeds the L1/L2 cache** (typically a few hundred KB), causing constant cache thrashing. SIMD instructions execute in sub-nanoseconds, but the CPU waits milliseconds for data from main RAM. Solutions: cache-blocking (tiling) or GPU offloading.

8 Phase 3: GPU & Hybrid Architecture (Day 41–70)

Phase 3 moves computation to the mobile GPU using OpenGL ES 3.1 compute shaders. The primary challenge is not the shader math itself, but the **memory transport overhead**.

Core Objectives:

- **Compute Shader Pipeline:** GLSL shaders for all 5 filters. Pixel data uploaded via SSBOs (Shader Storage Buffer Objects).
- **Readback Optimization:** `glReadPixels` is a severe pipeline staller. Pixel Buffer Objects (PBOs) pipeline the CPU↔GPU transfers to overlap frame preparation with rendering.
- **Hybrid Load Balancing:** Frame split horizontally — top 50% to NEON thread, bottom 50% to GPU. Requires careful synchronization to avoid idle-waiting.
- **Headless EGL Context:** Offscreen rendering context for compute shaders (no visible surface needed).

9 Phase 4: Final Tuning & Demo (Day 71–80)

Optimization Targets:

- **Thermal Stress Testing:** Map thermal throttling behavior during sustained Hybrid execution (5+ minutes).
- **Decoupled UI Rendering:** Separate metric calculations from the Android view rendering cycle to prevent UI jank.
- **Adaptive Mode (Stretch Goal):** Dynamically switch execution paths based on thermal status and battery level — GPU when cold, SIMD when hot, Baseline when critical.
- **Dashboard Polish:** Per-filter latency breakdown, thermal status indicator, cumulative speedup display.

10 Correctness Verification & Risk Management

Floating-point math on a GPU rounds differently than fixed-point arithmetic in a NEON register. We do not expect bit-perfect pixel matches across modes.

Verification Strategy:

- Pixel-wise comparison against the Kotlin baseline.
- Tolerance: ± 2 for SIMD (fixed-point truncation), ± 3 for GPU (single-precision float differences).
- Automated verification function runs on every mode switch.

Primary Risks:

- **NDK Build Fragility:** CMake configurations can break across machines. NDK version pinned, native codebase isolated.
- **Thermal Walls:** Aggressive OS downclocking during sustained GPU use. Mitigation: dynamic resolution scaling.
- **Memory Wall:** Gaussian blur's 8 MB temp buffer exceeds CPU cache. Mitigation: cache-blocking or GPU offloading.

End of Design Document — March 2026